

Curtin University – Department of Computing

Assignment Cover Sheet / Declaration of Originality

Complete this form if/as directed by your unit coordinator, lecturer or the assignment specification.

Last name:	Choo	Student ID:	23447635
Other name(s):	Jamie Chung Sheng		
Unit name:	Data Structure and Algorithm	Unit ID:	COMP1002
Lecturer / unit coordinator:	Ahlam Al-Dhamari	Tutor:	—
Date of submission:	28/5/2026	Which assignment?	(Leave blank if the unit has only one assignment.)

I declare that:

- The above information is complete and accurate.
- The work I am submitting is *entirely my own*, except where clearly indicated otherwise and correctly referenced.
- I have taken (and will continue to take) all reasonable steps to ensure my work is *not accessible* to any other students who may gain unfair advantage from it.
- I have *not previously submitted* this work for any other unit, whether at Curtin University or elsewhere, or for prior attempts at this unit, except where clearly indicated otherwise.

I understand that:

- Plagiarism and collusion are dishonest, and unfair to all other students.
- Detection of plagiarism and collusion may be done manually or by using tools (such as Turnitin).
- If I plagiarise or collude, I risk failing the unit with a grade of ANN ("Result Annulled due to Academic Misconduct"), which will remain permanently on my academic record. I also risk termination from my course and other penalties.
- Even with correct referencing, my submission will only be marked according to what I have done myself, specifically for this assessment. I cannot re-use the work of others, or my own previously submitted work, in order to fulfil the assessment requirements.
- It is my responsibility to ensure that my submission is complete, correct and not corrupted.

Signature: _____ Date of signature: 28/5/2026

(By submitting this form, you indicate that you agree with all the above text.)

ZipRide Dispatch System Report

Author: Jamie Choo Chung Sheng

Unit: COMP1002 Data Structure and Algorithms

1. Introduction

The ZipRide Dispatch System is a simplified ride-booking and dispatch system implemented with fundamental data structures and algorithms in JAVA. This project aims to model the efficient management of location, passengers, drivers, pickup scheduling and route optimisation in a transportation dispatch system. This project combines several data structures and algorithms in a single system. A graph data structure was used to store the locations of the cities and the roads between them. Graphs were traversed and cycles were detected using Breadth-First Search (BFS) and Depth-First Search (DFS), and Dijkstra's shortest path algorithm was used to determine the shortest driving time between locations. To handle the records of passengers and drivers efficiently, hash tables with linear probing were used. These structures enable insertion, search, deletion and collision handling operations. A max heap was used to schedule the Pickup requests. The system considers passenger membership level and estimated pick up time in calculating priority scores for requests. Requests with higher priority are dispatched first. Sorting algorithms have also been programmed without using the built-in sorting functions. Pickup records were sorted by the estimated pickup time (ascending) using merge sort and quick sort. Benchmark testing was performed on various data sets of varying sizes and conditions, and the algorithms were compared. The project illustrates the ways various data structures and algorithms can be combined in an actual system of application to ensure good performance and scalability.

This is my Menu

```
=====
ZIPRIDE DISPATCH SYSTEM
=====
1. Display Graph
2. BFS Traversal
3. DFS Cycle Detection
4. Dijkstra Shortest Path
5. Display Passenger Hash Table
6. Display Driver Hash Table
7. Create Pickup Request
8. Dispatch Highest Priority Request
9. Display Heap
10. Merge Sort Pickup Records
11. Quick Sort Pickup Records
12. Sorting Benchmark
0. Exit

Enter choice: █
```

2. Module 1 – Graph Algorithms

2.1 Graph Representation

The transportation network in the ZipRide Dispatch System was implemented using an adjacency list graph structure. Each location in the city is represented as a vertex, while each road connection between locations is represented as an edge with an associated travel time weight.

An adjacency list representation was selected because it is memory efficient for sparse graphs. Since not every location is directly connected to every other location, adjacency lists reduce unnecessary memory usage compared to an adjacency matrix.

The graph module supports:

- Adding locations
- Adding roads
- Displaying graph connections
- Breadth-First Search (BFS)
- Depth-First Search (DFS) cycle detection
- Dijkstra shortest path algorithm

```
ZipRide City Graph:
CurtinUniversity-> Airport(17 mins), Restaurant(9 mins), ShoppingMall(11 mins)
Airport-> CurtinUniversity(17 mins), Beach(12 mins)
Restaurant-> CurtinUniversity(9 mins), Hospital(6 mins)
Hospital-> Restaurant(6 mins), ShoppingMall(7 mins), Park(10 mins)
ShoppingMall-> CurtinUniversity(11 mins), Hospital(7 mins), Park(4 mins)
PetShop-> Beach(9 mins), Park(5 mins)
Park-> PetShop(5 mins), Hospital(10 mins), ShoppingMall(4 mins)
Beach-> Airport(12 mins), PetShop(9 mins)
IsolatedNode->
```

2.2 Breadth-First Search (BFS)

Breadth-First Search was implemented to traverse the transportation network level by level starting from a selected source location.

The BFS algorithm uses a queue to ensure that neighbouring locations are visited before moving deeper into the graph. A visited array is also used to prevent revisiting nodes repeatedly.

BFS is useful for:

- Exploring connected locations
- Checking graph connectivity
- Traversing the transportation network systematically

Time Complexity

For a graph with V vertices and E edges:

$O(V+E)$

This complexity occurs because each vertex and edge is visited at most once.

```
BFS from CurtinUniversity:  
Level 0: CurtinUniversity  
Level 1: Airport, Restaurant, ShoppingMall  
Level 2: Hospital, Park, Beach  
Level 3: PetShop
```

2.3 Depth-First Search (DFS) Cycle Detection

Depth-First Search was implemented recursively to detect cycles within the transportation network.

DFS explores as deeply as possible along each branch before backtracking.

During traversal, the algorithm checks whether a visited neighbour is not the parent node. If this occurs, a cycle exists in the graph.

Cycle detection is important because transportation systems often contain multiple routes connecting the same locations. Detecting cycles helps verify graph correctness and identify alternative travel routes.

The DFS implementation also displays the locations involved in detected cycles.

Time Complexity

$O(V+E)$

DFS visits every vertex and edge once during traversal.

```
Cycle detected in the graph.  
Cycle members: CurtinUniversity -> Airport -> Beach -> PetShop -> Park -> Hospital -> Restaurant -> CurtinUniversity  
Cycle detected in the graph.
```

2.4 Dijkstra Shortest Path Algorithm

Dijkstra's algorithm was implemented to calculate the shortest travel time between two locations in the transportation network.

The algorithm repeatedly selects the unvisited location with the smallest current distance value and updates neighbouring distances if shorter paths are found.

This algorithm is essential in the ZipRide Dispatch System because estimated pickup time (T) is used for:

- Selecting the nearest available driver
- Calculating pickup priority scores
- Benchmark dataset generation for sorting analysis

The shortest path calculation directly integrates with later modules such as heap scheduling and sorting.

Time Complexity

$O(V^2)$

This complexity is based on the array-based implementation used in the project.

```
Shortest path from Airport to Hospital:  
Airport -> CurtinUniversity -> Restaurant -> Hospital  
Total driving time: 32 minutes
```

3. Module 2 – Hash Tables

3.1 Hash Table Design

Two separate hash tables were implemented in the ZipRide Dispatch System:

- Passenger Hash Table

```
Passenger Hash Table:
Index | State | Record
-----
0 | Used | Passenger ID: 1001, Name: Jamie, Pcikup: , Tier: 1
1 | Used | Passenger ID: 1012, Name: Samuel, Pcikup: , Tier: 2
2 | Used | Passenger ID: 1023, Name: Elson, Pcikup: , Tier: 3
3 | Used | Passenger ID: 1034, Name: Allyn, Pcikup: , Tier: 1
4 | Used | Passenger ID: 1045, Name: Kelson, Pcikup: , Tier: 4
5 | Empty |
6 | Empty |
7 | Empty |
8 | Empty |
9 | Empty |
10 | Empty |
Load factor: 0.45454545454545453
```

- Driver Hash Table

```
Driver Hash Table:
Index | State | Record
-----
0 | Used | Driver ID 2012, Name: Benjamin, Location: CurtinUniversity, Status: Available
1 | Used | Driver ID 2023, Name: Jasper, Location: Hospital, Status: Busy
2 | Used | Driver ID 2034, Name: Davin, Location: Restaurant, Status: Available
3 | Used | Driver ID 2045, Name: Macus, Location: ShoppingMall, Status: Offline
4 | Empty |
5 | Empty |
6 | Empty |
7 | Empty |
8 | Empty |
9 | Empty |
10 | Used | Driver ID 2001, Name: Kelven, Location: Airport, Status: Available
Load factor: 0.45454545454545453
```

Both hash tables were implemented using open addressing with linear probing for collision resolution. The purpose of the hash tables is to provide efficient storage and retrieval of passenger and driver records. Fast searching is important in a ride dispatch system because passenger and driver information must be accessed quickly during scheduling and dispatch operations.

Each hash table entry stores:

- Unique ID
- Name
- Location
- Additional status information

The implementation supports:

- Insert operation
- Search operation
- Delete operation
- Collision handling
- Load factor calculation

3.2 Linear Probing

Linear probing was selected as the collision handling strategy.

When a collision occurs, the algorithm checks the next available position sequentially until an empty slot is found.

I choose to use Linear probing and not chaining because. It is simpler to implement. It provides good cache performance and can avoid the additional memory overhead required by chaining

However, linear probing may suffer from primary clustering when many collisions occur.

3.3 Hash Function

This function converts the passenger or driver ID into an array index.

Prime table sizes were used to reduce clustering and improve key distribution.

3.4 Load Factor

The load factor measures how full the hash table becomes.

A high load factor increases collisions and reduces performance efficiency.

Therefore, maintaining a reasonable load factor is important for efficient searching and insertion operations.

3.5 Time Complexity

Average Case	
Operation	Complexity
Insert	$O(1)$
Search	$O(1)$
Delete	$O(1)$

Worst Case	
Operation	Complexity
Insert	$O(n)$
Search	$O(n)$
Delete	$O(n)$

Worst-case performance occurs when excessive clustering causes many sequential probes.

3.6 Driver Availability Management

The Driver Hash Table stores driver availability status such as:

- Available
- Busy
- Offline

During pickup request creation, the system searches only for the nearest available driver. Once a request is dispatched, the driver's status is updated to "Busy".

This integration demonstrates how the hash table module interacts with both the graph and heap scheduling modules.

4. Module 3 – Heap-Based Pickup Scheduling

4.1 Purpose of the Heap Module

The heap module is used to manage pickup requests based on priority. In a ride dispatch system, some pickup requests should be handled before others depending on passenger priority and driver distance.

The heap module integrates results from previous modules:

- Passenger membership tier is retrieved from the Passenger Hash Table.
- Available drivers are retrieved from the Driver Hash Table.
- Estimated pickup time is calculated using Dijkstra's shortest path algorithm from the Graph module.

4.2 Max Heap Selection

I used max heap because the pickup request with the highest priority value should be dispatched first. Since the priority formula produces larger values for more urgent requests, placing the largest priority at the root of the heap allows the system to quickly access the next request to dispatch.

The main heap operations implemented are:

- insert()
- peek()
- extractPriority()
- trickleUp()
- trickleDown()

After each insertion and extraction, the heap array is printed to show how the heap structure changes.

4.3 Priority Calculation

Each pickup request uses the priority formula:

$$\text{Priority} = (6 - M) + 1000 / T$$

where:

- M = passenger membership tier
- T = estimated pickup time from the nearest available driver to the passenger pickup location

A lower membership tier number represents a higher passenger status. For example, tier 1 passengers receive higher priority than tier 5 passengers.

The second part of the formula rewards shorter pickup times. If the driver is closer to the passenger, the value of $1000 / T$ becomes larger, giving that request a higher priority score. If the estimated pickup time is 0 because the driver is

already at the pickup location, the system uses 1 instead to avoid division by zero.

4.4 Driver Selection Strategy

When a pickup request is created, the system first searches for the passenger record from the Passenger Hash Table. Then, it retrieves all drivers with the status “Available” from the Driver Hash Table.

For each available driver, the system uses the Graph module to calculate the shortest travel time from the driver’s current location to the passenger’s pickup location. The driver with the lowest estimated pickup time is selected as the nearest driver.

After the nearest driver is found, a PickupRequest object is created and inserted into the max heap.

4.5 Heap Operations

Insert Operation

When a pickup request is inserted, it is first placed at the end of the heap array. The trickleUp operation is then used to compare the new request with its parent. If the new request has a higher priority than its parent, the two records are swapped. This continues until the heap property is restored.

Extract Priority Operation

The extractPriority operation removes the root of the heap, which contains the highest-priority pickup request. The last element in the heap is moved to the root, and trickleDown is used to restore the heap property.

```
Extracted highest priority request:
Passenger: Jamie | Driver: Benjamin | Pickup Time: 0 mins | Priority: 1005.0
Current Heap Array:
[Empty]
Dispatched request:
Passenger: Jamie | Driver: Benjamin | Pickup Time: 0 mins | Priority: 1005.0
Driver status updated to Busy.
```

4.6 Time Complexity

Operation	Time Complexity
insert()	$O(\log n)$
peek()	$O(1)$
extractPriority()	$O(\log n)$

The logarithmic complexity occurs because heap operations only move along the height of the binary heap.

5. Module 4 – Sorting Algorithms

5.1 Purpose of the Sorting Module

The sorting module was implemented to organise pickup records based on estimated pickup time in ascending order. Sorting allows the system to analyse completed pickup requests efficiently and compare ride performance across different requests.

The sorting key used in this project is: EstimatedPickupTime (T)
where smaller values appear earlier in the sorted list.

5.2 Merge Sort

Merge sort was implemented using the top-down recursive approach.

The algorithm works by repeatedly dividing the array into smaller halves until each subarray contains only one element. The smaller subarrays are then merged back together in sorted order.

The merge process compares elements from two sorted subarrays and places the smaller element into a temporary array until all elements are merged.

I selected Merge Sort because. It guarantees $O(n \log n)$ performance. It is stable and it performs consistently regardless of input order. However, merge sort requires additional temporary memory during merging operations.

Time Complexity	
Case	Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$

Stability

Merge sort is a stable sorting algorithm because equal elements maintain their original relative order after sorting.

```
Before Merge Sort:
Jamie | Driver: Kelven | Time: 17 mins | Priority: 63.6
Samuel | Driver: Benjamin | Time: 17 mins | Priority: 60.8
Elson | Driver: Davin | Time: 6 mins | Priority: 153.0
Allyn | Driver: Kelven | Time: 1 mins | Priority: 171.6
Kelson | Driver: Benjamin | Time: 11 mins | Priority: 87.3
```

```
After Merge Sort:
Allyn | Driver: Kelven | Time: 1 mins | Priority: 171.6
Elson | Driver: Davin | Time: 6 mins | Priority: 153.0
Kelson | Driver: Benjamin | Time: 11 mins | Priority: 87.3
Jamie | Driver: Kelven | Time: 17 mins | Priority: 63.6
Samuel | Driver: Benjamin | Time: 17 mins | Priority: 60.8
```

```
Merge Sort Time: 10044 ns
```

5.3 Quick Sort

Quick sort was implemented using recursive partitioning.

The algorithm selects a pivot element and rearranges the array so that:

- smaller elements are placed on the left side
- larger elements are placed on the right side

The same process is then recursively applied to the left and right partitions.

The implementation in this project uses the rightmost element as the pivot.

The rightmost pivot strategy was selected because it is simple to implement. It integrates well with recursive partitioning and it reduces implementation complexity. However, poor pivot selection may cause unbalanced partitions and reduce performance efficiency.

Time Complexity	
Case	Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n^2)$

The worst case occurs when the array is already sorted or reversed and poor pivot selection creates highly unbalanced partitions.

Stability

Quick sort is not a stable sorting algorithm because equal elements may change relative order during partitioning.

```
Before Quick Sort:
Jamie | Driver: Ali | Time: 15 mins | Priority: 71.6
Alex | Driver: Ben | Time: 8 mins | Priority: 129.0
Sarah | Driver: David | Time: 20 mins | Priority: 53.0
Daniel | Driver: Ali | Time: 6 mins | Priority: 171.6
Emily | Driver: Ben | Time: 12 mins | Priority: 87.3

After Quick Sort:
Daniel | Driver: Ali | Time: 6 mins | Priority: 171.6
Alex | Driver: Ben | Time: 8 mins | Priority: 129.0
Emily | Driver: Ben | Time: 12 mins | Priority: 87.3
Jamie | Driver: Ali | Time: 15 mins | Priority: 71.6
Sarah | Driver: David | Time: 20 mins | Priority: 53.0

Quick Sort Time: 24489 ns
```

5.4 Benchmark

Merge sort showed consistent performance across all dataset conditions because the divide-and-conquer strategy always splits the array evenly. Its performance was not significantly affected by input order.

Quick sort generally performed faster on random datasets because partitioning operations required fewer comparisons and swaps. However, performance degraded on reversed datasets because the rightmost pivot strategy produced highly unbalanced partitions, increasing recursion depth.

Nearly sorted datasets also affected quick sort performance depending on pivot placement, while merge sort remained relatively stable.

Overall:

- Merge sort provided more consistent performance
- Quick sort provided faster average performance on random datasets
- Quick sort was more sensitive to poor pivot selection

```
===== PICKUP RECORD SORTING BENCHMARK =====
Size | Condition | Merge Sort(ns) | Quick Sort(ns)
-----
100 | Random | 44856 | 127101
100 | Nearly Sorted | 37173 | 98288
100 | Reversed | 37806 | 112300
500 | Random | 59838 | 220950
500 | Nearly Sorted | 60199 | 241566
500 | Reversed | 41055 | 150162
1000 | Random | 85364 | 793333
1000 | Nearly Sorted | 78385 | 1655773
1000 | Reversed | 84542 | 990140
```

6. Conclusion

The ZipRide Dispatch System successfully demonstrated the integration of multiple data structures and algorithms into a single transportation dispatch application. The project combined graph traversal, shortest path calculation, hash tables, heap scheduling, and sorting algorithms to simulate how a ride dispatch system manages passengers, drivers, and pickup requests efficiently.

The graph module allowed the system to model transportation routes and calculate shortest travel times using Dijkstra's algorithm. BFS and DFS were also implemented to support graph traversal and cycle detection.

Hash tables provided efficient storage and retrieval of passenger and driver records using linear probing for collision handling. The heap scheduling module allowed high-priority pickup requests to be dispatched efficiently using a max heap structure.

Merge sort and quick sort were implemented without using built-in sorting methods. From the testing, we can see that merge sort showed more stable performance across all datasets, while quick sort performed faster on average random datasets but was more sensitive to pivot selection and input order.

From this assignment, I have deeply learned how data structure algorithms are important and what roles they play. It also demonstrated the importance of algorithm efficiency and appropriate data structure selection in real-world software systems.

7. References

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

GeeksforGeeks. (2026). Heap Data Structure. Retrieved from <https://www.geeksforgeeks.org/dsa/heap-data-structure/>

Programiz. (2026). Graph Data Structure. Retrieved from <https://www.programiz.com/dsa/graph>

Baeldung. (2026). Dijkstra's Shortest Path Algorithm in Java. Retrieved from <https://www.baeldung.com/java-dijkstra>

Lecture Notes and Tutorial Materials for COMP1002 Data Structures and Algorithms.

8. Appendix

Before starting the menu, the information is already inserted:

```
Location added: CurtinUniversity
Location added: Airport
Location added: Restaurant
Location added: Hospital
Location added: ShoppingMall
Location added: PetShop
Location added: Park
Location added: Beach
Location added: IsolatedNode
Road added: CurtinUniversity <-> Airport (17mins)
Road added: CurtinUniversity <-> Restaurant (9mins)
Road added: CurtinUniversity <-> ShoppingMall (11mins)
Road added: Restaurant <-> Hospital (6mins)
Road added: ShoppingMall <-> Hospital (7mins)
Road added: Airport <-> Beach (12mins)
Road added: Beach <-> PetShop (9mins)
Road added: PetShop <-> Park (5mins)
Road added: Park <-> Hospital (10mins)
Road added: ShoppingMall <-> Park (4mins)
```

```
Inserted passenger 1001 at index 0
Collision at index 0 for passenger 1012
Inserted passenger 1012 at index 1
Collision at index 0 for passenger 1023
Collision at index 1 for passenger 1023
Inserted passenger 1023 at index 2
Collision at index 0 for passenger 1034
Collision at index 1 for passenger 1034
Collision at index 2 for passenger 1034
Inserted passenger 1034 at index 3
Collision at index 0 for passenger 1045
Collision at index 1 for passenger 1045
Collision at index 2 for passenger 1045
Collision at index 3 for passenger 1045
Inserted passenger 1045 at index 4
Inserted driver 2001 at index 10
Collision at index 10 for driver 2012
Inserted driver 2012 at index 0
Collision at index 10 for driver 2023
Collision at index 0 for driver 2023
Inserted driver 2023 at index 1
Collision at index 10 for driver 2034
Collision at index 0 for driver 2034
Collision at index 1 for driver 2034
Inserted driver 2034 at index 2
Collision at index 10 for driver 2045
Collision at index 0 for driver 2045
Collision at index 1 for driver 2045
Collision at index 2 for driver 2045
Inserted driver 2045 at index 3
```